Assignment 1 solution

1. 【考点：计算机性能指标与程序执行】某计算机主频为 2 GHz。程序 P 在该计算机上的原指令数为 10^9 条，指令分类及原 CPI 如下：ALU 指令占 50%（CPI=1），分支指令占 20%（CPI=3），访存指令占 30%（CPI=2）。优化编译器后，程序 P 的 ALU 指令数减少了 20%，分支指令数不变，但由于增加了地址计算，访存指令的平均 CPI 变为 2.5。关于编译优化后的程序 P，下列说法正确的是（ ）

   A. CPU 执行时间缩短了 2.94%，总体 MIPS 提升
   B. CPU 执行时间延长了 2.94%，总体 CPI 增加
   C. 总指令数减少了 10%，CPU 执行时间缩短
   D. 总体 CPI 降低，CPU 执行时间不变

   【答案】：B
   【解析】：
   计算原程序执行参数：
   原总指令数 $N_{orig} = 10^9$
   原总时钟周期数 $C_{orig} = 10^9 \times (0.5 \times 1 + 0.2 \times 3 + 0.3 \times 2) = 1.7 \times 10^9$ 个
   原执行时间 $T_{orig} = (1.7 \times 10^9) / 2 \times 10^9$ Hz = 0.85 s
   原平均 $CPI_{orig} = (1.7 \times 10^9) / 10^9 = 1.7$
   计算优化后程序执行参数：
   ALU 指令数 = $0.5 \times (1 - 0.2) \times 10^9 = 0.4 \times 10^9$
   分支指令数 = 0.2×10^9
   访存指令数 = 0.3×10^9
   优化后总指令数 $N_{opt} = (0.4 + 0.2 + 0.3) \times 10^9 = 0.9 \times 10^9$（指令数减少了 10%）
   优化后总时钟周期数 $C_{opt} = 0.4 \times 10^9 \times 1 + 0.2 \times 10^9 \times 3 + 0.3 \times 10^9 \times 2.5 = 1.75 \times 10^9$ 个
   优化后执行时间 $T_{opt} = (1.75 \times 10^9) / 2 \times 10^9$ Hz = 0.875 s
   优化后平均 $CPI_{opt} = (1.75 \times 10^9 / 0.9 \times 10^9 \approx 1.944$（CPI 显著增加）
   比较结果：
   执行时间变化比：(0.875 - 0.85) / 0.85 ≈ 2.94%，即执行时间延长了 2.94%（性能下降）
   尽管指令总数减少，但由于 CPI 较高的访存指令占比上升且其 CPI 变大，导致总体 CPI 增加，最终执行时间变长。故 B 正确

2. 【考点：计算机硬件组成与指令执行周期】某单总线 16 位 CPU 中，所有指令均为单字长 16 位，主存按字节编址。主存读/写一次需 1 个总线周期。当前程序计数器 PC 内容为 `2000H`。CPU 执行一条双操作数加法指令 `ADD (R0), R1`，其功能为 ((R0)) + (R1) → (R0)。已知 R0 内容为 `3000H`，R1 内容为 `4000H`，`3000H` 单元的内容为 `0005H`。在从取指令开始到该指令执行结束的过程中，总线传输的次数以及最终 PC 的值分别为（ ）

   A. 3 次，`2002H`
   B. 2 次，`2002H`
   C. 3 次，`2001H`
   D. 4 次，`2002H`

   【答案与】：A
   【解析】：
   取指阶段：
   CPU 从 PC（`2000H`）指向的主存单元读取 16 位（2 字节）指令，发生 1 次总线读传输
   指令字长 16 位（2 字节），按字节编址，故 PC 自动递增 2，最终 PC = 2000H + 2 = 2002H

执行阶段：

操作数一取值：根据源寻址方式 (R0)，主存地址为 `3000H`，读取该单元数据 `0005H`，发生 1 次总线读传输

操作数二取值：R1 为寄存器寻址，直接从寄存器读取 `4000H`，无需访问总线

ALU 计算：0005H + 4000H = 4005H

结果写回：目的寻址方式为 (R0)，将结果 `4005H` 写入主存 `3000H` 单元，发生 1 次总线写传输

汇总：总线传输总次数为 1 + 1 + 1 = 3 次，最终 PC = 2002H

3. 【考点：数据表示与 C 语言类型转换/移位】在 32 位计算机系统（采用补码表示整数，小端字节序）中，执行以下 C 语言代码：

```
short x = -32768;
int y = (int)x;
unsigned int z = (unsigned int)y;
z = z >> 1;
```

执行完毕后，变量 `z` 的十六进制表示为（ ）

A. `0x7FFF 8000`

B. `0x7FFF C000`

C. `0xFFFF C000`

D. `0x0000 4000`

【答案】：B

【解析】：

`short x = -32768;`：

16 位有符号整数，-32768 的补码表示为 `0x8000`（二进制 `1000 0000 0000 0000`）

`int y = (int)x;`：

16 位有符号数扩展为 32 位有符号数，执行符号扩展（高位补符号位 `1`）

`y` 的补码为 `0xFFFF 8000`（二进制 `1111 1111 1111 1111 1000 0000 0000 0000`）

`unsigned int z = (unsigned int)y;`：

强转为无符号整数，位模式保持不变，`z` 的十六进制仍为 `0xFFFF 8000`

`z = z >> 1;`：

由于 `z` 是 `unsigned int`，右移执行逻辑右移（高位补 `0`）

二进制右移 1 位：`0111 1111 1111 1111 1100 0000 0000 0000`

转换为十六进制为 `0x7FFF C000`

4. 【考点：运算电路与 ALU 标志位生成】设 8 位补码加/减法运算器中，输入为 8 位补码 A 和 B，控制信号 Sub = 1 表示执行减法 A - B。加法器输入端口 X = A，Y = $\overline{B}$，进位输入 C_{in} = Sub = 1。已知输入 A = 80H，B = 01H。在执行 A - B 后，ALU 生成的标志位 OF（溢出）、SF（符号）、ZF（零）、CF（进/借位，定义 CF = $C_{out} \oplus$ Sub）的值分别是（ ）

A. OF=1, SF=0, ZF=0, CF=0

B. OF=1, SF=1, ZF=0, CF=1

C. OF=0, SF=0, ZF=0, CF=0

D. OF=1, SF=0, ZF=0, CF=1

【答案】：A

【解析】：

数据与加法器输入：

A = 80H = $1000\ 0000_2$（真值 -128）

B = 01H = $0000\ 0001_2$（真值 +1）

取反 $\overline{B}$ = $1111\ 1110_2$

C_{in} = 1

硬件实际加法过程：

$$\begin{array}{rl} 1000\ 0000_2 & (A) \\ +\ 1111\ 1110_2 & (\bar{B}) \\ +\ 0000\ 0001_2 & (C_{in}) \\ \hline =\ 1\ 0111\ 1111_2 & (\text{结果为 } 0111\ 1111_2 = 7\text{FH}) \end{array}$$

进位分析：

最高数值位（Bit 6）向符号位（Bit 7）的进位 C_6 = 0（低 7 位计算 000 0000 + 111 1110 + 1 = 111 1111_2，无进位）

符号位（Bit 7）产生的进位 $C_{out} = C_7$ = 1（1 + 1 + 0 = 2 → 产生进位 1，本位为 0）

标志位判定：

OF（溢出标志） = $C_7 \oplus C_6 = 1 \oplus 0 = 1$。（实际真值 -128 - 1 = -129，超出 8 位补码范围 [-128, 127]，发生下溢）

SF（符号标志） = 结果的最高位 = 0

ZF（零标志） = 结果是否全 0 = 0

CF（借位标志） = $C_{out} \oplus$ Sub = $1 \oplus 1 = 0$。（无符号视角：128 - 1 = 127 ≥ 0，无借位，故 CF=0）

5. 【考点：补码一位乘法 Booth 算法】采用 Booth 算法（补码一位乘法）计算 x × y，已知乘数 y = -7，用 5 位补码表示为 y = 11001_2（包含 1 位符号位）。被乘数 x = -11，5 位补码为 $[x]_{补}$ = 10101_2。在 Booth 算法的完整乘法运算过程中，涉及加法/减法（加 $[-x]_{补}$）操作的总次数以及最终算术右移的总次数分别为（ ）

A. 加/减法 3 次，右移 5 次
B. 加/减法 4 次，右移 5 次
C. 加/减法 3 次，右移 4 次
D. 加/减法 2 次，右移 5 次

【答案】：A

【解析】：

参数准备：

$[x]_{补} = 10101_2$，$[-x]_{补} = 01011_2$

乘数扩展位 y_{-1} = 0，初始乘数状态 $y_4\ y_3\ y_2\ y_1\ y_0\ y_{-1} = 110010_2$

5 位补码乘法需循环 5 步（5 次右移）

逐步推演：

第 1 步：检查 $y_0\ y_{-1} = 10_2$ → 减 x（即加 $[-x]_{补}$）。（第 1 次加/减操作），随后算术右移 1 位

第 2 步：移位后 $y_1\ y_0 = 01_2$ → 加 x（加 $[x]_{补}$）。（第 2 次加/减操作），随后算术右移 1 位

第 3 步：移位后 $y_2\ y_1 = 00_2$ → 仅算术右移 1 位（不执行加/减）

第 4 步：移位后 $y_3\ y_2 = 10_2$ → 减 x（加 $[-x]_{补}$）。（第 3 次加/减操作），随后算术右移 1 位

第 5 步：移位后 $y_4\ y_3 = 11_2$ → 仅算术右移 1 位（不执行加/减）

总结：总共执行了 3 次加/减法操作，5 次算术右移

6. 【考点：定点数除法——加减交替法】关于原码加减交替法（不恢复余数法）实现定点小数除法 x / y（|x| < |y|），下列说法中错误的是（ ）

A. 第一步先用被除数绝对值 $[|x|]_{补}$ 减去除数绝对值 $|y|]_{补}$，若余数为正，说明发生溢出
B. 运算过程中，若当前步相减得到的余数为负，则该步商 0，下一步将余数左移一位后加上 [|y|]补
C. 若进行了 n 位小数的除法，在求得 n 位数值位商的过程中，共需要进行 n+1 次加/减法操作
D. 若最后一步（第 n+1 次）得到的余数为负，无需恢复余数，直接将商的末位强制置 1 即可得到正确余数

【答案】：D

【解析】：

A 正确：定点小数要求 $|x| < |y|$。第一步计算 $[|x|]_{补} - [|y|]_{补}$，若结果为正，说明 $|x| \geq |y|$，商 ≥ 1，定点小数溢出

B 正确：加减交替法的核心规则：余数为正商 1，下步左移减除数；余数为负商 0，下步左移加除数

C 正确：确定符号/溢出判断算第 1 次，求 n 位小数商需 n 次，总共 n+1 次加/减法

D 错误：若最后一步算出的余数为负，虽然商的位数已确定，但为了得到正确的余数（用于后续取模或精准余数保留），必须加上 $[|y|]_{补}$ 来恢复余数（商的末位不需要修改）。因此"无需恢复余数"的说法错误

7. 【考点：IEEE 754 浮点数编码与转换】在 IEEE 754 单精度浮点数标准（32 位）中，二进制比特串 `0xBF40 0000` 所表示的十进制真值是（ ）

A. -0.75

B. -1.5

C. -0.375

D. -0.625

【答案】：A

【解析】：

解析比特串：

`0xBF40 0000` 展开为二进制：`1 01111110 10000000000000000000000`

拆解各字段：

符号位 S = 1（负数）

阶码 E = 0111 1110_2 = 126

尾数 M = 1000...0_2，实际尾数（含隐藏位 1）为 1.1_2 = 1.5

计算真值：

移码偏移量为 127，实际阶码 e = E - 127 = 126 - 127 = -1

真值 $V = (-1)^S \times (1 + M) \times 2^e = -1 \times 1.5 \times 2^{-1} = -0.75$

8. 【考点：浮点数加/减运算过程与舍入】设浮点数格式为：阶码 4 位（包含 1 位阶符，补码表示），尾数 5 位（包含 1 位数符，补码表示）。已知两浮点数 $x = 2^{-1} \times (-0.1101_2)$，$y = 2^1 \times (+0.1001_2)$。采用补码进行 x + y 计算，经过对阶、尾数求和、规格化及舍入（采用"0 舍 1 入"法）后，最终的浮点数运算结果为（ ）

A. $2^1 \times (+0.1100_2)$

B. $2^0 \times (+0.1100_2)$

C. $2^1 \times (+0.0110_2)$

D. 发生阶码上溢

【答案】：B

【解析】：

转换补码表示：

x: 阶码 $E_x = -1 = 1111_2$，尾数 $M_x = -0.1101_2 = 1.0011_2$

y: 阶码 $E_y = +1 = 0001_2$，尾数 $M_y = +0.1001_2 = 0.1001_2$

对阶（小阶向大阶看齐）：

阶差 $\Delta E = E_x - E_y = -1 - 1 = -2$

x 的阶码增大到 +1（0001_2），尾数 M_x 右移 2 位（算术右移，高位补 1），保留附加位：

右移 1 位：1.1001(1)

右移 2 位：1.1100(11)，即 $M'_x = 1.1100_2$，附加位为 11

尾数求和：

$$\begin{array}{rl} & 1.1100(11)_2 \\ + & 0.1001(00)_2 \\ \hline = & 0.0101(11)_2 \end{array}$$

规格化（左规）：

当前尾数 $0.0101(11)_2$ 非规格化数（双符号位为 00，数值最高位为 0）

左移 1 位：尾数变为 $0.1011(10)_2$，阶码减 1，变为 $0001_2 - 1 = 0000_2$（阶码 0）

舍入（0 舍 1 入）：

附加位为 10，最高附加位为 1，向尾数最低位进 1：

$0.1011_2 + 0.0001_2 = 0.1100_2$

舍入后尾数依然为规格化形式

最终结果：阶码为 0000_2（真值 0），尾数为 0.1100_2（真值 $+0.1100_2$），即 $2^0 \times (+0.1100_2)$。

故 B 正确

9. 【考点：C 语言数据类型转换与精度损失】假设在遵循 IEEE 754 标准的 32 位计算机系统中，执行以下 C 语言代码：

```
int i = 0x7FFFFFFF;
float f = (float)i;
int j = (int)f;
```

下列关于 `i`、`f`、`j` 关系的描述中，正确的是（ ）

A. `i == j`，因为 `float` 的表示范围远大于 `int`，转换不会丢失精度

B. `i != j`，因为 `int` 转为 `float` 时尾数发生舍入，且 `f` 强转回 `int` 时发生溢出

C. `f` 的二进制尾数精确保留了 `i` 的所有有效数字

D. `i` 转为 `float` 时发生阶码上溢

【答案】：B

【解析】：

`int` 转 `float`（`i` → `f`）：

`i = 0x7FFFFFFF`（即 $2^{31} - 1 = 2147483647$），二进制为 31 个 `1`

IEEE 754 单精度 `float` 的尾数位数为 24 位（1 位隐藏位 + 23 位小数位）

将 31 位有效数字压缩进 24 位尾数，必须进行舍入（丢失低 7 位精度）

$2^{31} - 1$ 舍入到最近的可表示浮点数，结果为 $2^{31} = 2147483648.0$

`float` 转 `int`（`f` → `j`）：

`f` 的真值为 2147483648（2^{31}）

32 位有符号 `int` 所能表示的最大正数为 $2^{31} - 1$

2^{31} 超出了 `int` 的上限，强转导致整型溢出（在绝大多数 GCC/x86 实现中，结果会变为 `0x80000000` 即 -2147483648）

结论：`i` 与 `j` 显然不相等，原因在于 `int` → `float` 的尾数精度舍入以及 `float` → `int` 的数值溢出

10. 【考点：指令周期与取指微操作】某计算机的数据通路包含通用寄存器、程序计数器 PC、指令寄存器 IR、主存地址寄存器 MAR、主存数据寄存器 MDR 以及 ALU。主存按字节编址，指令字长为 32 位（4 字节）。在取指令阶段，控制单元（CU）发出的微操作序列如下：

    1. (PC) → MAR，Read
    2. M(MAR) → MDR，(PC) + 4 → PC
    3. (MDR) → IR

    下列关于上述微操作及其硬件执行逻辑的叙述，错误的是（ ）

A. 微操作 1 和微操作 2 不能在同一个时钟周期内完成
B. 微操作 2 中，M(MAR) → MDR 与 (PC) + 4 → PC 可以并行执行
C. 若主存按字（32 位）编址，则微操作 2 中的 PC 更新动作应修改为 (PC) + 1 → PC
D. 指令译码（Decode）操作与微操作 3 在同一个时钟周期内并行发生

【答案】：D
【解析】：
A 正确：微操作 1 刚将地址送入 MAR 并发出读命令，主存建立数据需要读存延迟，因此读出数据到 MDR（微操作 2）必须在后续的时钟周期进行
B 正确：从主存读数据到 MDR 使用的是主存与数据总线，而 PC 加 4 使用的是专用的 PC 增量器或 ALU，两者的硬件资源无冲突，可以并行执行
C 正确：指令长 32 位。若按字节编址，每条指令占 4 个地址单元，故 PC 加 4；若按字（32 位）编址，每条指令仅占 1 个地址单元，故 PC 加 1
D 错误：微操作 3 将指令从 MDR 打入指令寄存器 IR。只有当指令完全装入 IR 后，译码器才能对 IR 中的操作码字段进行逻辑译码。因此指令译码必须发生在微操作 3 完成之后，不能并行发生

11. 【考点：计算机性能指标】某计算机的主频为 2.0 GHz。某测试程序包含 10^6 条指令，其中 50% 为 ALU 指令，平均每条指令执行需 1 个时钟周期；30% 为访存指令，平均需 3 个时钟周期；20% 为分支指令，平均需 2 个时钟周期。现欲升级该计算机，采用新的微架构使得主频提升至 3.0 GHz，但由于流水线加深，使得所有类别指令的平均 CPI 都统一增加了 0.2 个时钟周期。关于升级前后的性能指标，下列说法正确的是?

A. 升级前，该程序在计算机上的 CPU 执行时间为 1.8 ms
B. 升级前，该计算机执行该程序的运算速度为 2000 MIPS
C. 升级后，该程序在新计算机上的 CPU 执行时间约为 0.67 ms
D. 升级后，由于指令的平均 CPI 变大，系统执行该程序的整体性能下降

【答案】：C
【解析】：
升级前 CPI = 0.5 × 1 + 0.3 × 3 + 0.2 × 2 = 0.5 + 0.9 + 0.4 = 1.8
升级前 CPU 执行时间 = (指令条数 × CPI) / 主频 = (10^6 × 1.8) / (2.0×10^9) = 0.9×10^{-3} s = 0.9 ms。故选项 A 错误
升级前 MIPS = 主频 / (CPI × 10^6) = 2000 MHz / 1.8 ≈ 1111 MIPS。故选项 B 错误
升级后 CPI = 1.8 + 0.2 = 2.0
升级后 CPU 执行时间 = (10^6 × 2.0) / (3.0×10^9) = (2 / 3) × 10^{-3} s ≈ 0.67 ms。故选项 C 正确
升级后的执行时间（0.67 ms）小于升级前的执行时间（0.9 ms），整体性能是提升的，故选项 D 错误

12. 【考点：加/减运算】某 16 位计算机中，ALU 采用补码进行加/减法运算。设有 16 位带符号整数 A = 0x7FFF， B = 0x8000。在执行 A - B 的运算后，ALU 生成的标志位 ZF（零标志）、SF（符号标志）、OF（溢出标志）和 CF（进位/借位标志，减法时视为借位标志）的状态分别是?

A. ZF=0, SF=0, OF=1, CF=0
B. ZF=0, SF=1, OF=1, CF=1
C. ZF=0, SF=1, OF=0, CF=1
D. ZF=0, SF=0, OF=1, CF=1

【答案】：B
【解析】：
在 16 位系统中，A = 0x7FFF 代表最大正整数 +32767；B = 0x8000 代表最小负整数 -32768
执行 A - B 即 32767 - (-32768) = 65535。作为带符号 16 位整数，结果超出了表示范围 [-32768, 32767]，发生了正溢出，因此 OF = 1
底层 ALU 计算为 A + (~ B + 1)，即 0x7FFF + 0x7FFF + 1 = 0xFFFF
结果 0xFFFF 各位全为 1，非零，故 ZF = 0。最高位（符号位）为 1，故 SF = 1
对于减法，无符号数 A (32767) < B (32768)，必然产生借位。在加法器中 CF = 最高位进位 $\oplus$ 1。加法运算进位为 0，异或 1 得到借位 CF = 1。因此选 B

13. 【考点：浮点数的表示】考虑 C 语言中的数据类型转换，假设 `float` 采用 IEEE 754 单精度浮点数标准表示，`int` 为 32 位带符号整数。对于以下 `int` 类型的变量 `x`，在严格执行 `float y = (float)x; int z = (int)y;` 操作后，必定会导致 `x != z` （即发生精度丢失）的 `x` 的初始值是?

A. 0x00FF FFFF
B. 0x0100 0000
C. 0x0100 0001
D. 0x8000 0000

【答案】：C
【解析】：
IEEE 754 单精度浮点数包含 1 位符号位、8 位阶码和 23 位尾数（隐含 1 位最高位 1，实际有 24 位有效精度）
A 项 0x00FF FFFF 值为 2^{24}-1，对应二进制连续 24 个 1，正好填满有效尾数，无精度丢失
B 项 0x0100 0000 值为 2^{24}，即 1.0×2^{24}，尾数全 0，无精度丢失
C 项 0x0100 0001 值为 2^{24}+1，对应的二进制为 10000000000000000000000001，从最高位 1 到最低位 1 跨越了 25 位。单精度尾数只有 24 位，最后一位 1 会被截断/舍入，必定导致精度丢失
D 项 0x8000 0000 为 -2^{31}，即 -1.0×2^{31}，尾数全 0，无精度丢失

14. 【考点：乘/除运算】在计算机的算术逻辑单元中，乘法和除法通常采用特定的硬件算法来实现。关于整数乘除法运算的基本原理，下列说法错误的是?

A. 补码乘法（Booth 算法）支持带符号数的直接相乘，由于引入了加减交替操作，在遇到连续的 0 或连续的 1 时能够减少实际的加法或减法操作次数
B. 在原码恢复余数除法中，如果当前步减去除数后得到的余数为负，则商寄存器当前位上 0，并将余数加上除数以恢复到原值
C. 在补码加减交替除法（非恢复余数法）中，若最终余数与被除数异号，说明最后一步“不够减”，必须直接以该余数作为最终正确余数，不可再做任何调整
D. 无论采用原码乘法还是补码乘法，两个 n 位数相乘，为了防止溢出并保留完整精度，其乘积至少需要使用 2n 位的寄存器来保存

【答案】：C
【解析】：
补码加减交替法（非恢复余数法）中，如果最终余数与被除数异号，说明最后一步出现了“不够减”的情况。此时，虽然商已经是正确的，但余数是错误的。必须再进行一次“加上或减去除数”的操作，使其与被除数同号，从而恢复出真正的最终余数。因此 C 选项中“不可再做任何调整”的说法是错误的。A、B、D 描述的原理均正确

15. 【考点：计算机系统的工作原理】根据冯·诺依曼计算机“存储程序”的工作方式，以及高级语言程序向机器语言转换、执行的过程，下列描述中存在逻辑错误的是?

A. 计算机能自动工作的基本前提是预先编制好程序，并将程序和数据以同等地位存放在存储器中，由 CPU 按照首地址按序读取执行
B. 高级语言程序通常必须经过预处理、编译、汇编和链接等步骤，其中汇编器（Assembler）的输出结果一般是可重定位目标文件，而不是能直接运行的绝对机器代码
C. 存储器中的数据和指令在物理形态上都是 0/1 序列，CPU 主要是通过指令周期的不同阶段（如取指周期取出的是指令，执行周期取出的是数据）来进行区分
D. 为了提高内存利用率和程序执行效率，现代计算机在将高级语言转换为机器语言时，编译器能够自动决定并动态修改处理器底层硬件的操作码（Opcode）映射规则

【答案】：D
【解析】：
处理器底层的指令系统结构（ISA）及其操作码（Opcode）的硬件映射规则是固化在 CPU 硬件设计中（如硬布线控制器或微程序控制器）的。编译器（软件）只能根据给定的 ISA 将高级语言翻译

成对应的机器码，无法也不可能去动态修改硬件的操作码映射规则。因此选项 D 存在严重的逻辑错误，符合题意要求。A、B、C 均为计算机系统概述中的标准正确概念

16. **34**．B

$2^{15}=32768$，i = 32768 + 9 = 8000H + 9H = 8009H，32 位有符号数 i 的机器数为 0000 8009H。将 32 位有符号数 i 强制转换为 16 位有符号数 si，直接保留机器数的末 16 位即可，因此 si 的机器数为 8009H，真值为$-2^{15}+9=-32759$。将 16 位有符号数 si 强制转换为 32 位有符号数 j，采用符号扩展（si 的符号为 1，因此高位补 1），j 的机器数变为 FFFF 8009H，对应的真值为−32759。

	int i	short si	int j
机器数	0000 8009H	8009H	FFFF 8009H
真值	32777	−32759	−32759

17. **35**．D

阵列乘法器中的所有部分积同时产生并组成一个阵列，运用多操作数相加就能得到最终的积，因此可在一个时钟周期内完成。用 ALU 和移位器实现的乘运算通常采用串行的乘法算法，需要多个时钟周期才能完成。当一个乘数是常数时，编译器可将乘运算优化为若干移位和加/减运算指令。两个变量的乘运算可通过移位和加法等指令循环实现，选项 D 错误。

18. **51**．C

表示整数时，相同位数的浮点型的精度不如整型，因此在能满足 α、β 的取值范围的前提下，应优先选择整型；否则，才选择浮点型。32 位补码整数的表示范围为$-2^{31}\sim2^{31}-1$，满足 α 的取值范围，因此 α 应采用 32 位整数。双精度浮点数的精度比单精度浮点数的更高，$-2^{40}\sim2^{40}$超出了单精度浮点数可以表示的精度（由尾数位数+隐含位决定），因此 β 应采用双精度浮点数。

19. **34**．D

在 32 位系统中，short 通常为 16 位有符号整数。−32767 的 16 位补码为 1000 0000 0000 0001。将其赋值给 32 位 unsigned int 时，先按**符号扩展**转换为 32 位有符号整数 1111 1111 1111 1111 1000 0000 0000 0001，再以无符号方式解释，其值为$2^{32}-(2^{15}-1)=2^{32}-2^{15}+1$。

20. **35**．D

要求计算 $x-y$，即$[x]_{补}+[-y]_{补}$。首先求$[-y]_{补}$：对$[y]_{补}$按位取反得 1000 1010，再加 1，得 1000 1011 = 8BH。然后计算$[x]_{补}+[-y]_{补}$ = A3H + 8BH = 1010 0011 + 1000 1011 = (1)0010 1110，忽略进位后，结果为 0010 1110 = 2EH，对应十进制数 46。判断溢出：参与运算的两个操作数符号位均为 1（负数），而结果符号位为 0（正数），表明发生了溢出，因此 OF = 1。

21. **44**．D

根据 IEEE 754 单精度浮点数格式，机器数 4730 0000H 对应的二进制数为 0, 100 0111 0, 011 0000 0000 0000 0000 0000。符号位为 0，表示正数；8 位阶码 10001110 的十进制值为 142，减去偏置 127，得到实际指数为 15；23 位尾数为 011 0000 0000 0000 0000 0000，规格化形式隐含了前导 1，故有效数字为 1.011（二进制），即十进制数 1.375，因此 x 的值为1.375×2^{15}。

22. **45**．C

首先分析结构体 record 的内存布局：int id 占 4 字节；char name[10]占 10 字节，为使后续 int salary 对齐到 4 字节边界，需要在 name 后填充 2 字节，使 salary 从 4 的整数倍偏移处开始。因此结构体总大小为 4 + 10 + 2 + 4 = 20 字节。employee[1]的首地址为 0000A0B0H + 20 = 0000A0C4H，id 为其第一个成员，起始地址即为 0000A0C4H。已知 id 的值为 12345678H，在小端方式下，字节按 78H、56H、34H、12H 的顺序存放，故 56H 位于 0000A0C5H。

23.

04. 【解答】

1）因为 134 = 128 + 6 = 1000 0110B，所以 x 的机器数为 1000 0110B，因此 R1 的内容为 86H。246 = 255 – 9 = 1111 0110B，所以 y 的机器数为 1111 0110B，$x-y$ = 1000 0110 + 0000 1010 = (0)1001 0000，括号中为加法器的进位，因此 R5 的内容为 90H。$x+y$ = 1000 0110 + 1111 0110 = (1)0111 1100，括号中为加法器的进位，因此 R6 的内容为 7CH。

2）m 的机器数与 x 的机器数相同，皆为 86H = 1000 0110B，解释为有符号整数 m（用补码表示）时，其值为−111 1010B = −122。$m-n$ 的机器数与 $x-y$ 的机器数相同，皆为 90H = 1001 0000B，解释为有符号整数 k1（用补码表示）时，其值为−111 0000B = −112。

3）能。n 位加法器实现的是模 2^n 无符号整数加法运算。对于无符号整数 a 和 b，$a+b$ 可以直接用加法器实现，而 $a-b$ 可用 a 加 $-b$ 的补数实现，即 $a-b=a+[-b]_{补} \pmod{2^n}$，所以 n 位无符号整数加减运算都可在 n 位加法器中实现。

因为有符号整数用补码表示，补码加减运算公式为 $[a+b]_{补}=[a]_{补}+[b]_{补} \pmod{2^n}$，$[a-b]_{补}=[a]_{补}+[-b]_{补} \pmod{2^n}$，所以 n 位有符号整数加减运算都可在 n 位加法器中实现。

4）有符号整数加减运算的溢出判断规则为：若加法器的两个输入端（加法）的符号相同，且不同于输出端（和）的符号，则结果溢出，或加法器完成加法操作时，若次高位（最高数位）的进位和最高位（符号位）的进位不同，则结果溢出。

最后一条语句执行时会发生溢出。因为 1000 0110 + 1111 0110 = (1)0111 1100，括号中为加法器的进位，根据上述溢出判断规则可知结果溢出。或者，因为两个有符号整数均为负数，它们相加之后，结果小于 8 位二进制所能表示的最小负数。

24. **05.** 【解答】

1）乘法运算可以通过加法和移位来实现。编译器可以将乘法运算转换为一个循环代码段，在循环代码段中通过比较、加法和移位等指令实现乘法运算。

2）控制逻辑的作用是控制循环次数，控制加法和移位操作。

3）a 最长，c 最短。对于 a，需要用循环程序段实现乘法操作，因此需要反复执行很多条指令，而每条指令都需要取指令、译码、取数、执行并保存结果，所以执行时间很长；对于 b 和 c，都只需用一条乘法指令实现乘法操作，不过 b 中的乘法指令需要多个时钟周期才能完成，而 c 中的乘法指令可在一个时钟周期内完成，所以 c 的执行时间最短。

4）当 $n=32, x=2^{31}-1, y=2$ 时，有符号整数和无符号整数乘法指令得到的 64 位乘积都是 0000 0000 FFFF FFFEH。int 型的表示范围为 $[-2^{31}, 2^{31}-1]$，所以函数 imul()的结果溢出；unsigned int 型的表示范围为 $[0, 2^{32}-1]$，所以函数 umul()的结果不溢出。对于无符号整数乘法，若乘积高 n 位全为 0，即使低 n 位全为 1 也正好是 $2^{32}-1$，不溢出，否则溢出。

注意，无论是无符号数还是有符号数，用 $2n$ 位来表示两个 n 位整数的相乘结果都不会溢出，因为 $2n$ 位可以完整地存储两个 n 位整数的乘积。但是，若只用低 n 位来表示结果，则可能溢出。因此，要保证低 n 位转换成的真值与 $2n$ 位转换成的真值相等才算是不溢出。对于无符号数，只要高 n 位全为 0，就不会溢出，因为高 n 位在转换成真值后不会影响低 n 位的值。对于有符号数，要考虑符号位的影响。当结果是正数时，符号位为 0，要求高 n 位也全为 0，且低 n 位的最高位也为 0（否则正数变负数）。当结果是负数时，符号位为 1，要求高 n 位也全为 1，且低 n 位的最高位也为 1（否则负数变正数）。因此，在有符号数的情况下，高 $n+1$ 位相同表示不溢出。

25. **05.** 【解答】

1）i 和 n 是 unsigned 型，因此“i <= n−1”是无符号数比较，n = 0 时，n − 1 的机器数为全 1，值是 $2^{32}-1$，为 unsigned 型可表示的最大数，条件“i <= n − 1”永真，因此出现死循环。

若 i 和 n 改为 int 型，则不会出现死循环。

因为“i <= n−1”是有符号整数比较，当 n = 0 时，n − 1 的值是−1，当 i = 0 时，条件“i <= n−1”不成立，此时退出 for 循环。

2）f1(23)与 f2(23)的返回值相等。$f(23) = 2^{23+1} - 1 = 2^{24} - 1$，其二进制形式是 24 个 1。int 型数占 32 位，没有溢出。float 型数有 1 个符号位，8 个指数位，23 个底数位，23 个底数位可以表示 24 位的底数，所以两者返回值相等。

f1(23)的机器数是 00FF FFFFH，f2(23)的机器数是 4B7FFFFFH。

显而易见，前者是 24 个 1，即 $0000\ 0000\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111_{(2)}$，后者的符号位是 0，指数位为 $23 + 127_{(10)} = 1001\ 0110_{(2)}$，底数位是 $111\ 1111\ 1111\ 1111\ 1111\ 1111_{(2)}$。

3）当 n = 24 时，f(24) = 1 1111 1111 1111 1111 1111 1111 B，而 float 型数只有 24 位有效位，舍入后数值增大，所以 f2(24)比 f1(24)大 1。

4）显然 f(31)已超出了 int 型数的表示范围，用 f1(31)实现时得到的机器数为 32 个 1，作为 int 型解释时其值为−1，即 f1(31)的返回值为−1。

因为 int 型最大可表示的数是 0 后面加 31 个 1，因此使 f1(n)的返回值与 f(n)相等的最大 n 值是 30。

5）IEEE 754 标准使用“阶码全 1、尾数全 0”表示无穷大。f2 的返回值为 float 型，机器数 7F80 0000H 对应的值是+∞。当 n = 126 时，$f(126) = 2^{127} - 1 = 1.1\ldots1 \times 2^{126}$，对应的阶码为 127 + 126 = 253，尾数部分舍入后阶码加 1，最终阶码为 254，是 IEEE 754 单精度格式表示的最大阶码。因此使 f2 结果不溢出的最大 n 值为 126。

当 n = 23 时，f(23)为 24 位 1，float 型数有 24 位有效位，所以不需要舍入，结果精确。因此使 f2 获得精确结果的最大 n 值为 23。

26. 【考点：计算机性能指标】某处理器的时钟频率为 2GHz。一段原始的高级语言程序经旧编译器处理后，生成了 3 类机器指令（A、B、C），其指令条数分别为 2M（$1M = 10^6$）、1M 和 1M，且这三类指令的 CPI 分别为 1、2、3。现在采用了一款全新的编译器进行优化，优化后的机器代码中，将原来所有的 C 类指令替换为了 1.5M 条 A 类指令，A、B 类指令的其他部分保持不变。关于此次优化的计算机性能指标分析，下列选项中正确的是（ ）

A. 优化后程序的指令总数增加了，因此 CPU 执行时间必然变长

B. 优化后程序的平均 CPI 降低，MIPS 提高，CPU 执行时间缩短了 0.75ms

C. 优化后处理器的 MIPS 得到提升，这说明“计算机系统的基本组成”中硬件的主频通过动态加速得到了提高

D. 优化后程序的吞吐量下降，响应时间变短

【答案】：B

【解析】：

计算原始性能：

总指令数 = 2M + 1M + 1M = 4M 条

总时钟周期数 = 2M × 1 + 1M × 2 + 1M × 3 = 7M 个周期

CPU 执行时间 = 7M / 2GHz = 3.5×10^{-3} 秒 = 3.5ms

平均 CPI = 7M / 4M = 1.75

MIPS = 4M / 3.5ms ≈ 1142

计算优化后性能：

新的指令组合为：A 类指令 3.5M（2M + 1.5M），B 类指令 1M，C 类指令 0

新总指令数 = 4.5M 条

新总时钟周期数 = 3.5M × 1 + 1M × 2 = 5.5M 个周期

新 CPU 执行时间 = 5.5M / 2GHz = 2.75ms。缩短了 3.5 - 2.75 = 0.75ms

新平均 CPI = 5.5M / 4.5M ≈ 1.22

新 MIPS = 4.5M / 2.75ms ≈ 1636

A 错：由于高 CPI 指令被低 CPI 指令替代，指令总数虽然增加，但总时钟周期数减少，CPU 执行时间反而缩短

B 对：符合上述计算推导

C 错：MIPS 提升是因为指令组合的变化导致平均执行速度变快，硬件主频 2GHz 并没有发生改变

D 错：执行时间缩短代表响应时间变短，单位时间内能执行的任务量增加，因此吞吐量是上升的，而不是下降

27. 【考点：运算方法和运算电路】在 32 位计算机系统中，整数采用补码表示。现有两个变量 `x = 0x80000000` 和 `y = 0x7FFFFFFF`，当算术逻辑部件（ALU）执行减法指令 `x - y` 时，底层加法器生成的标志位 CF（进位/借位标志）、OF（溢出标志）、SF（符号标志）和 ZF（零标志）的状态分别是（ ）

A. CF=0, OF=1, SF=0, ZF=0
B. CF=1, OF=1, SF=1, ZF=0
C. CF=0, OF=0, SF=1, ZF=0
D. CF=1, OF=0, SF=0, ZF=1

【答案】：A
【解析】：

题目中 `x` 为 32 位最小负数，`y` 为 32 位最大正数。ALU 在执行减法 `x - y` 时，底层加法器实际执行的是 `x + (~y) + 1`
`y` 的机器数为 `0x7FFFFFFF`，按位取反 `~y` 为 `0x80000000`
加法器执行：`0x80000000`（`x`）+ `0x80000000`（`~y`）+ `1`（减法控制输入）= `0x00000001`，且最高位向外产生了一个进位 `Cout = 1`
SF（符号标志）：结果 `0x00000001` 的最高位（符号位）为 0，因此 SF=0
ZF（零标志）：结果不为全 0，因此 ZF=0
CF（进/借位标志）：对于减法，借位标志 `CF = Cout ⊕ 1`（无符号减法时，发生借位 CF=1）。本题 `Cout=1`，所以 `CF = 1 ⊕ 1 = 0`。从逻辑上看，无符号数 `0x80000000` 大于 `0x7FFFFFFF`，小减大才借位，这里大减小，无需借位，CF=0
OF（溢出标志）：判定带符号数溢出。负数 `x` 减去正数 `y`，理论上应该得到一个更小的负数（符号位应为 1）。但实际结果的符号位变为了 0，发生了典型的“负减正得正”溢出错误，因此 OF=1
综合得出 CF=0, OF=1, SF=0, ZF=0，故选 A

28. 【考点：浮点数的表示与运算】根据 IEEE 754 单精度浮点数（32位）标准，默认采用“就近舍入，向偶数舍入”模式。在 C 语言程序中，执行以下关系运算表达式，其判定结果为 `true`（即成立）的是（ ）

A. `(float)16777216 + 1.0f == (float)16777216`
B. `(float)16777216 + 2.0f == (float)16777216`
C. `(float)16777216 + 3.0f == (float)16777216 + 2.0f`
D. `(float)8388608 + 1.0f == (float)8388608`

【答案】：A
【解析】：

解析 16777216 (2^{24})：其 IEEE 754 单精度表示的指数部分为 127 + 24 = 151，尾数部分（隐藏最高位 1）的 23 位全为 0。此时该浮点数阶码下的最小精度（1 ULP）相当于 $2^{24} \times 2^{-23} = 2^1 = 2.0$
A 项分析：在进行加法对阶时，`1.0f`（即 1.0×2^0）的尾数需要右移 24 位。其隐藏的“1”正好落在 23 位尾数之后的第 24 位（保护位 Guard bit），此时附加位为 `1 0 0`，代表恰好处于两个可表示浮点数的正中间（0.5 ULP）。根据“向偶数舍入”规则，由于当前尾数最低位为 0（已经是偶数），系统将舍弃这个附加位，发生截断。因此加 1.0 后的结果依然是 16777216，A 成立
B 项分析：加 `2.0f`，刚好是 1 ULP。右移 23 位，加在了尾数的最低有效位上，没有精度丢失，结果是 16777218，不等于原值，B 错
C 项分析：加 `3.0f` 时（1.5 ULP），对阶后附加位为 `1 1 0`，超过了 0.5 的阈值，必然向上进位。结果会变为 16777216 + 4.0，而非 + 2.0，C 错
D 项分析：`8388608` 是 2^{23}，其 1 ULP 精度恰好为 $2^{23} \times 2^{-23} = 2^0 = 1.0$。因此加 `1.0f` 不会丢失精度，结果变为 8388609，不等于原值，D 错

29. 【考点：数据的表示和运算】高级语言程序在向机器语言程序转换以及被 CPU 执行的过程中，深度依赖硬件对数据的表示和底层运算电路的实现。下列关于此过程的描述，正确的是（ ）

A. 在执行 `int a = -1; unsigned int b = 1; if(a > b)` 这一结构时，由于 `a` 是有符号数，编译器生成的机器指令会默认采用带符号比较指令，因此条件判定为假
B. 执行 `short x = -1; unsigned int y = x;` 时，高级语言向机器语言转换的规则是：先按目标类型（unsigned）的要求进行零扩展，最终 `y` 的机器数为 `0x0000FFFF`
C. 现代计算机中，无符号整数的加法和带符号整数的加法在底层共用同一个“加法器”电路核心，系统仅依靠生成的特定标志位（如 CF 和 OF）结合数据类型来区分不同的溢出逻辑
D. 冯·诺依曼计算机的“存储程序”工作方式意味着程序指令和数据在主存中是严格物理隔离存储的，因此 CPU 依靠物理地址段就能自然区分取出的二进制序列是指令还是数据

【答案】：C
【解析】：

A 错：在 C 语言等高级语言程序的转换中，当带符号数与无符号数进行关系运算时，带符号数会被隐式转换为无符号数处理。`-1` 对应的机器码被解释为无符号数 `0xFFFFFFFF`，这是一个极大的正数，因此 `a > b` 的判定实际上为真
B 错：不同字长、不同符号类型之间的数据转换，首先根据原数据类型的符号属性决定如何扩展。`x` 是带符号的 `short`，因此首先进行符号扩展到 32 位（变为 `0xFFFFFFFF`），然后再将其解释为 `unsigned int`，最终 `y` 的值是 `0xFFFFFFFF`，而非零扩展的 `0x0000FFFF`
C 对：无论是“无符号整数的表示和运算”还是“带符号整数的表示和运算”，底层的算术逻辑部件（ALU）均使用同一个硬件加法器。加法器本身不区分数据是否有符号，而是同时生成 CF、OF 等标志位，由后续指令根据当前被处理数据的逻辑类型去检查对应的标志位（无符号看 CF，有符号看 OF）
D 错：“存储程序”工作方式的核心是将指令和数据同等对待，统一存储在同一个主存中，硬件上并未进行物理隔离。CPU 是通过指令执行周期的不同阶段（如取指阶段取出的是指令，执行阶段按地址取出的往往是操作数数据）来区分指令和数据的

30. 1. 计算机性能指标计算

总指令条数 N = 4M + 1M + 3M + 2M = 10×10^6 条
总时钟周期数 = $(4 \times 1) + (1 \times 2) + (3 \times 3) + (2 \times 4) \times 10^6 = 2.3 \times 10^7$ 个周期（即 23×10^6 个）
CPU 执行时间 = 总时钟周期数 / 主频 = $(23 \times 10^6) / (2 \times 10^9)$ Hz = 1.15×10^{-2} s = 11.5 ms
平均 CPI = 总时钟周期数 / 总指令条数 = $(2.3 \times 10^7) / 10^7 = 2.3$
MIPS = 总指令条数 / (执行时间 $\times 10^6$) = $(10 \times 10^6) / (0.0115 \times 10^6) \approx 869.57$（或使用 主频 / (CPI $\times 10^6$) = 2000 / 2.3 ≈ 869.57）

2. 补码减法与状态标志位生成

① 二进制补码与结果：
`x = 0x80000000`，二进制补码：`1000 0000 0000 0000 0000 0000 0000 0000`
`y = 1`，二进制补码：`0000 0000 0000 0000 0000 0000 0000 0001`
减法转化：x - y = x + [~ y] + 1
[~ y] = $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1110_2$
底层加法器计算：`1000...0000` + `1111...1110` + `1` = `0111 1111 1111 1111 1111 1111 1111 1111`，最高位产生向外进位 $C_{out} = 1$
因此 `z` 的十六进制机器码为 `0x7FFFFFFF`

② 状态标志位推导：
ZF = 0：运算结果不为全 0
SF = 0：运算结果最高位（符号位）为 0
OF = 1：负数减正数（最小负数 `-2147483648` 减 `1`），理论上应得负数，但结果符号位变成了 `0`（正数），发生了带符号整数溢出

CF = 0：加法器最高位进位 C_{out} = 1。减法运算时 CF = $C_{out} \oplus 1 = 1 \oplus 1 = 0$（无符号数视角下 0x80000000 ≥ 1，无须借位）

3. 浮点数表示与类型转换

① `f` 的 IEEE 754 编码：

`x = 0x80000000` 对应的真值为 -2^{31}

数符 S：负数，故 S = 1

阶码 E：$-2^{31} = -1.0_2 \times 2^{31}$，实际阶码为 31。移码表示 E = 31 + 127 = 158 = $1001\ 1110_2$

尾数 M：隐藏最高位 `1` 后，剩余 23 位全为 `0`

拼合 32 位二进制：`1` | `1001 1110` | `000 0000 0000 0000 0000 0000` → `1100 1111 0000 0000 0000 0000 0000 0000`

转换为十六进制机器码为 `0xCF000000`

② 精度丢失分析：

不一定会发生精度丢失

原因：IEEE 754 单精度浮点数用 23 位显式尾数加上 1 位隐藏的最高位 `1`，能够精准表示 24 位 有效二进制位的整数。当 32 位 `int` 整数的实际有效位不超过 24 位（或低位全为 0，如 -2^{31}）时，可以精确转换为 `float` 而不丢失精度；只有当 `int` 的有效二进制位数超过 24 位时，多余的低位会被舍入，才导致精度丢失

4. 指令与数据的硬件区分机制

CPU 主要通过指令周期的不同执行阶段来区分从主存读取的二进制串是指令还是数据：

取指阶段（Fetch）：CPU 根据程序计数器（PC）提供的地址访问主存，此时读取到的二进制序列被硬件认定为机器指令，并送入指令寄存器（IR）中

执行阶段（Execute）：CPU 根据指令中的地址码访问主存，此时读取到的二进制序列被硬件认定为操作数/数据，并送入 ALU 或通用寄存器中